

Git y GitHub

Guía Completa para Principiantes

Última actualización: 30 de May de 2026

■ Una guía práctica para aprender control de versiones desde cero

Tabla de Contenidos

1. ¿Qué es Git?	Introducción a los conceptos básicos
2. Conceptos Fundamentales	Repositorios, commits y ramas
3. Instalación y Configuración	Cómo comenzar
4. Comandos Esenciales de Git	Los comandos más importantes
5. ¿Qué es GitHub?	Alojamiento en la nube
6. Flujo de Trabajo Básico	Del local a la nube
7. Colaboración en GitHub	Trabajando en equipo
8. Resolución de Conflictos	Cuando las cosas se complican

1. ¿Qué es Git?

Definición

Git es un sistema de control de versiones distribuido creado por Linus Torvalds en 2005. Permite rastrear cambios en archivos y coordinar el trabajo entre varias personas en un proyecto. Es la herramienta estándar en la industria del desarrollo de software.

¿Por qué usar Git?

- **Historial completo:** Guarda un registro de todos los cambios
- **Colaboración:** Permite que varios desarrolladores trabajen en el mismo proyecto
- **Ramas:** Desarrolla características aisladas sin afectar el código principal
- **Reversión:** Vuelve a versiones anteriores si algo sale mal
- **Seguridad:** Distribuido, no depende de un servidor central

Casos de uso

Git se usa en proyectos de software de cualquier tamaño: desde aplicaciones personales hasta proyectos de código abierto con miles de contribuidores como Linux, Python y React.

2. Conceptos Fundamentales

Repositorio (Repo)

Es una carpeta que contiene todos los archivos de tu proyecto y el historial completo de cambios. Puede ser local (en tu computadora) o remoto (en un servidor como GitHub).

Commit

Una "captura" de los cambios en tu código en un momento específico. Cada commit tiene un identificador único y una descripción del cambio realizado.

Rama (Branch)

Una línea de desarrollo independiente. Por defecto existe **main** (antes llamada **master**). Puedes crear nuevas ramas para trabajar en características sin afectar la rama principal.

Merge (Fusión)

Combina los cambios de una rama con otra. Por ejemplo, fusionar cambios de una rama de características hacia la rama principal cuando están listos.

Pull Request (PR)

Una solicitud para revisar y fusionar cambios. Es la base de la colaboración en GitHub, permitiendo que otros revisen tu código antes de ser integrado.

3. Instalación y Configuración

Instalación

Windows: Descarga desde git-scm.com y ejecuta el instalador

macOS: Usa Homebrew: `brew install git`

Linux: `sudo apt-get install git` (Debian/Ubuntu)

Configuración Inicial

Después de instalar, configura tu identidad:

```
$ git config --global user.name "Tu Nombre"
```

```
$ git config --global user.email "tu@email.com"
```

```
$ git config --list # Verifica la configuración
```

Usa `--global` para configurar todos tus repositorios, u omítelo para configurar solo el actual.

4. Comandos Esenciales de Git

Iniciando un repositorio

Crea un nuevo repositorio en tu carpeta actual:

```
$ git init
```

Clonando un repositorio

Descarga un repositorio existente:

```
$ git clone https://github.com/usuario/proyecto.git
```

Viendo el estado

Muestra cambios no guardados y rama actual:

```
$ git status
```

Agregando cambios (Staging)

Prepara cambios para el commit:

```
$ git add archivo.txt # Agrega un archivo  
$ git add . # Agrega todos los cambios
```

Haciendo commit

Guarda los cambios en el historial:

```
$ git commit -m "Descripción del cambio"
```

Viendo el historial

Muestra el registro de commits:

```
$ git log # Log detallado  
$ git log --oneline # Log resumido
```

Creando ramas

Crea y cambia entre ramas:

```
$ git branch nombre-rama # Crea rama  
$ git checkout nombre-rama # Cambia a rama  
$ git checkout -b nombre-rama # Crea y cambia
```

Fusionando ramas

Combina una rama con la actual:

```
$ git checkout main # Cambia a rama principal  
$ git merge nombre-rama # Fusiona la rama
```

Enviando cambios (Push)

Sube tus cambios al repositorio remoto:

```
$ git push origin nombre-rama
```

Descargando cambios (Pull)

Descarga cambios del repositorio remoto:

```
$ git pull origin nombre-rama
```

5. ¿Qué es GitHub?

Definición

GitHub es una plataforma en la nube que aloja repositorios Git. Proporciona herramientas adicionales para colaboración, como pull requests, issues y actions. Es el lugar más popular para alojar proyectos de código abierto.

Características principales

- **Alojamiento remoto:** Tu código está seguro en servidores de GitHub
- **Pull Requests:** Sistema de revisión de código
- **Issues:** Rastreo de bugs y características solicitadas
- **Acceso control:** Decide quién puede ver o editar tu código
- **Documentación:** Archivo README.md para describir tu proyecto
- **Comunidad:** Colabora con desarrolladores de todo el mundo

Creando una cuenta

1. Visita github.com
2. Haz clic en "Sign up"
3. Sigue los pasos para crear tu cuenta
4. Usa el mismo email que configuraste en Git

6. Flujo de Trabajo Básico

Paso a paso: Del local a GitHub

1. Crea un repositorio

```
$ git init mi-proyecto
```

2. Agrega tus archivos

```
$ git add .
```

3. Haz tu primer commit

```
$ git commit -m "Commit inicial"
```

4. Crea un repo en GitHub

Ve a github.com y crea un nuevo repositorio

5. Conecta al remoto

```
$ git remote add origin https://github.com/usuario/repo.git
```

6. Envía tus cambios

```
$ git push -u origin main
```

7. Colaboración en GitHub

Ciclo de colaboración

1. **Fork (Bifurcación):** Copia el repositorio a tu cuenta
2. **Clone:** Descarga a tu computadora
3. **Branch:** Crea una rama para tu característica
4. **Edita:** Haz cambios y commits
5. **Push:** Envía a tu fork en GitHub
6. **Pull Request:** Solicita que el proyecto original fusione tus cambios
7. **Revisión:** Los maintainers revisan y comentan
8. **Merge:** Tus cambios se fusionan al proyecto

Buenas prácticas

- **Commits atómicos:** Cada commit debe ser una unidad lógica de cambio
- **Mensajes claros:** "Agregar validación de email" es mejor que "Cambios"
- **Ramas descriptivas:** Usa nombres como feature/login-page, fix/bug-123
- **Pull requests enfocados:** No mezcles muchos cambios en uno
- **Revisar código:** Siempre revisa los cambios antes de mergear
- **Documentación:** Mantén actualizado el README.md

8. Resolución de Conflictos

¿Qué es un conflicto?

Ocurre cuando dos personas modifican las mismas líneas de un archivo y Git no puede decidir automáticamente qué versión es la correcta.

Cómo resolver un conflicto

- 1. Identifica el conflicto:** Git te avisará durante un merge
- 2. Abre el archivo:** Verás marcadores <<< === >>>
- 3. Decide qué mantener:** Elige una opción, ambas o personaliza
- 4. Elimina marcadores:** Borra los símbolos <<< === >>>
- 5. Commit:** Haz commit de la resolución

Ejemplo de conflicto

```
<<<<<< HEAD
```

```
var nombre = "Alice";
```

```
=====
```

```
var nombre = "Bob";
```

```
>>>>>> rama-feature
```

Resuelve eligiendo una opción, eliminando los marcadores, y haciendo commit.

Conclusión

Has aprendido los fundamentos de Git y GitHub. Estos conceptos son suficientes para comenzar tu viaje en el desarrollo de software. Recuerda que la práctica es la mejor manera de aprender. Crea proyectos pequeños, experimenta con ramas, y colabora con otros.

Próximos pasos

- **Crea un repositorio personal:** Practica con un proyecto pequeño
- **Contribuye a código abierto:** Busca proyectos para aprender
- **Aprende temas avanzados:** Rebasing, stashing, cherry-picking
- **Automatiza:** Explora GitHub Actions
- **Colabora:** Trabaja con otros en proyectos reales

¡Felicidades por completar esta guía! Ahora tienes las herramientas para colaborar en proyectos de software profesionales.